Санкт-Петербургский государственный университет

Кафедра системного программирования

Экспериментальное сравнение производительности разных решений на основе линейной алгебры

Максим Александрович Родионов, группа 23.Б15-мм

Санкт-Петербург
2026

Постановка задачи

В данной работе исследуется производительность реализаций на основе линейной алгебры из библиотек Spla и SuiteSparse:GraphBLAS + LAGraph следующих алгоритмов:

- BFS (Breadth-First Search)
- SSSP (Single-Source Shortest Path)
- TC (Triangle Counting)
- PR (PageRank)

Исследование направлено на ответ на следующий вопрос:

- При каких структурных характеристиках графа (размер, степень разреженности) архитектура GPU (Spla) обеспечивает лучшую производительность по сравнению с реализацией для CPU (LAGraph)?

Описание набора данных

- Критерии отбора
  - Источник: `SuiteSparse Matrix Collection`
  - Симметричность
  - Разнообразие структур: социальные сети, дорожные сети, синтетические графы
- Датасет

Граф	$\|V\|$	$\|E\|$	Avg Deg
coAuthorsCiteseer	227.3K	1.6M	7.2
coPapersDBLP	540.5K	30.5M	56.4
hollywood-2009	1.1M	112.8M	98.9
belgium_osm	1.4M	3.1M	2.2
roadNet-CA	2.0M	5.5M	2.8
rgg_n_2_22_s0	4.2M	60.7M	14.5
road_central	14.1M	33.9M	2.4

Определение стартовых вершин для BFS и SSSP

- **Стратегия выбора:** вершина со средней степенью в наибольшей компоненте связности
- **Параметры выбранных вершин**

Граф	LCC (%)	Индекс (0-based)	Степень
coAuthorsCiteseer	100%	4	4
coPapersDBLP	100%	21	34
hollywood-2009	93.8%	46	32
belgium_osm	100%	0	2
roadNet-CA	99.3%	0	3
rgg_n_2_22_s0	100%	1	14
road_central	100%	4	2

Описание эксперимента и адаптация бенчмарков (1/2)

Инфраструктура:

- За основу взята архитектура `spla-bench` (для этого сделан fork[1])
- Обновлены версии зависимостей: `LAGraph`, `Spla`, `SuiteSparse:GraphBLAS`
- Добавлена поддержка алгоритма PageRank
- Интегрирована функция выбора стартовой вершины

Унификация параметров запуска:

- **SSSP (LAGraph)**: установлен $\triangle = 1$ (Delta-Stepping)
  Обоснование: соответствует решению разработчиков для таких структур[2]
- **PageRank (Spla)**: установлен $\varepsilon = 10^{-4}$
  Обоснование: приведение к порогу сходимости, используемому в реализации LAGraph

[1] `https://github.com/RodionovMaxim05/spla-bench`
[2] `https://arxiv.org/abs/1911.06895`

Описание эксперимента и адаптация бенчмарков (2/2)

Модификации исходного кода LAGraph (Python-скрипты патчинга):

- **BFS**: включен режим замера *level-only* (без восстановления путей)
  Обоснование: аналогично реализации в Spla
- **Triangle Counting**: добавлена возможность принудительного отключения сортировки (`NoSort`)
  Цель: сравнение производительности стратегий *AutoSort* vs *NoSort* для выявления оптимальной конфигурации, хотя в Spla нет сортировки

Тестовый стенд и программное окружение

Аппаратная конфигурация:

- CPU: Intel Core i7-11370H (4 ядра, 8 потоков), частота зафиксирована на 3.3 GHz
- GPU: NVIDIA GeForce RTX 3050 Mobile (4 ГБ VRAM)
- RAM: 16 ГБ DDR4

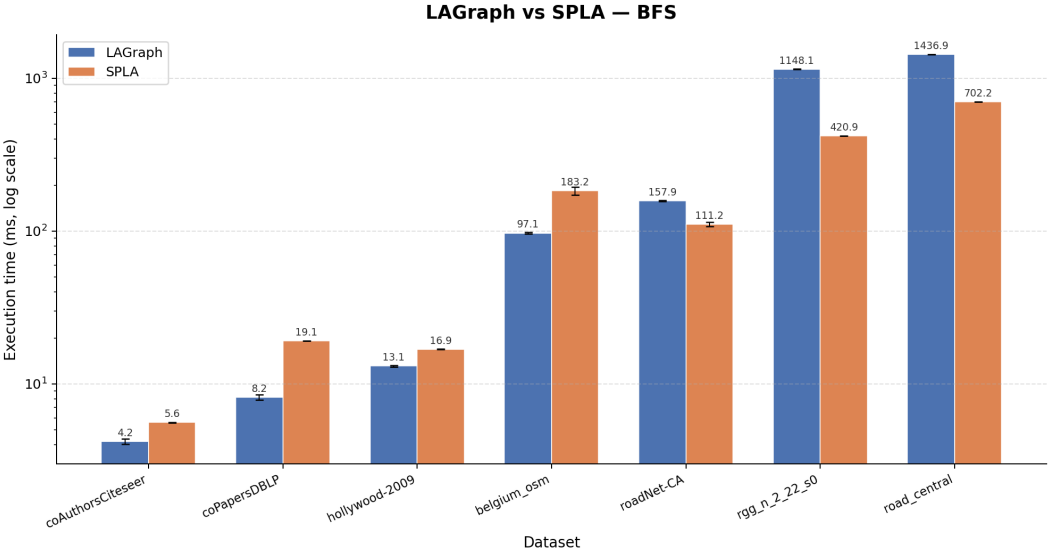
Программное обеспечение:

- ОС: Kubuntu 24.04 LTS
- Драйверы: NVIDIA Driver `v580.126.09`

Версии исследуемых библиотек:

- SuiteSparse:GraphBLAS: `v10.3.1`
- LAGraph: `v1.2.1`
- Spla: commit `b81d1e2`

Результаты BFS



BFS: результаты и объяснение (1/2)

Наблюдение: результаты делятся на две группы:

- **Spla быстрее** на `rgg`, `road_central`, `roadNet-CA` (ускорение 1.4–2.7×)
- **LAGraph быстрее** на `coAuthorsCiteseer`, `coPapersDBLP`, `hollywood`, `belgium_osm` (ускорение 1.3–2.3×)

Факторы, определяющие результат:

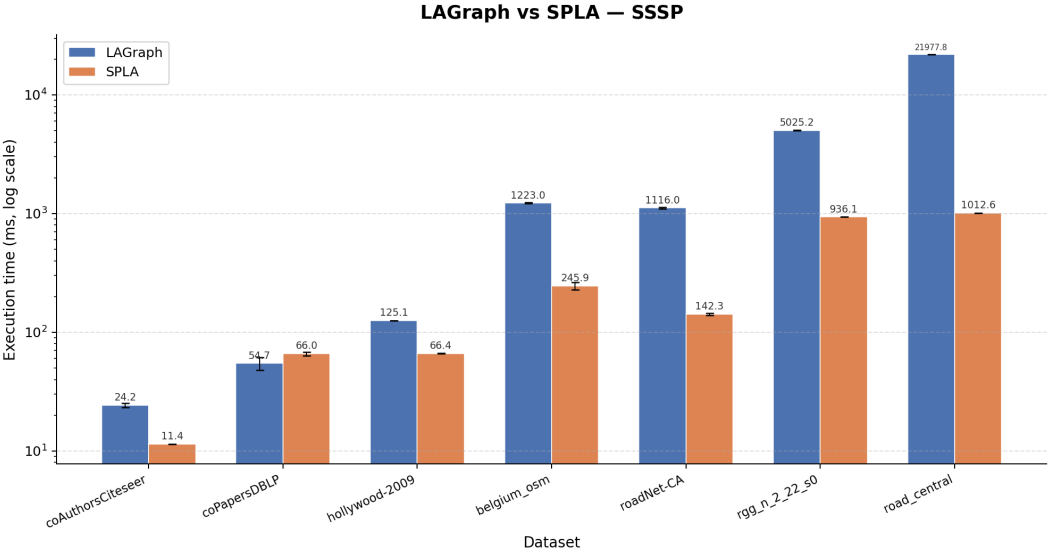
- **Spla**: медленная 1-ая итерация: в среднем замедление в 600 раз (от 120 до 1600 раз)
- **LAGraph**: узкое место — функция GB_quicksort_3_32_32_32

BFS: результаты и объяснение (2/2)

- **Spla** выигрывает там где итераций много, а frontier мал:

Граф	Кол-во итераций	Ср. размер frontier
coAuthorsCiteseer	25	9093
coPapersDBLP	16	33780
hollywood-2009	9	118792
belgium_osm	1459	988
roadNet-CA	555	3526
rgg_n_2_22_s0	1578	2658
road_central	3743	3762

Результаты SSSP



SSSP: результаты и объяснение (1/2)

Наблюдение: в отличие от BFS, Spla доминирует почти на всех графах:

- **Spla быстрее** на 6 из 7 графов (ускорение 1.9× – 21.7×)
- **LAGraph быстрее** только на `coPapersDBLP` (1.32×)

Ключевой фактор производительности LAGraph: накладные расходы на управление параллелизмом (OpenMP)

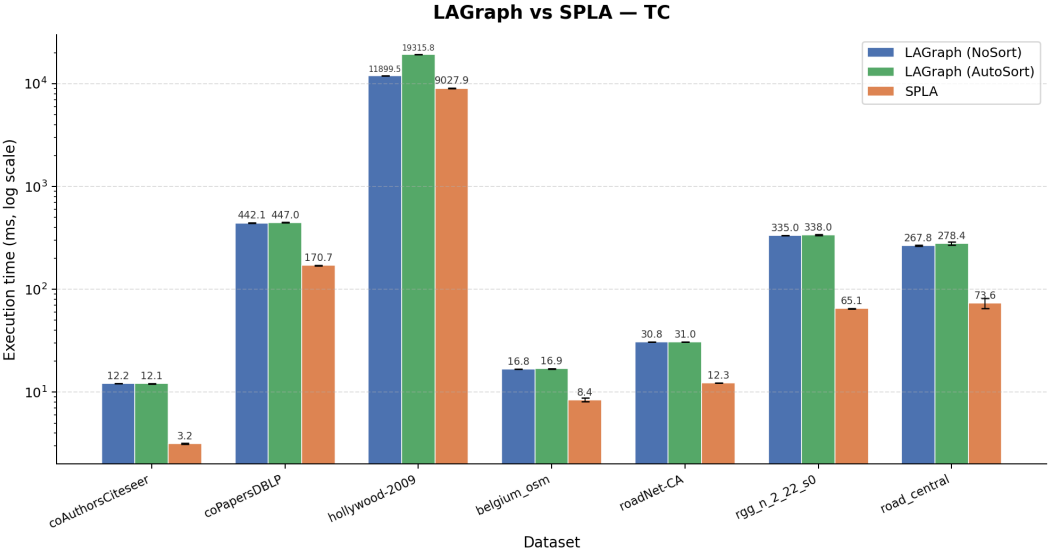
Граф	Доминирующая функция (perf)	Доля времени
coAuthorsCiteseer	`libgomp` (OpenMP runtime)	∼36%
roadNet-CA	`libgomp` (OpenMP runtime)	∼62%
road_central	`libgomp` (OpenMP runtime)	∼68%
coPapersDBLP	`GB_quicksort_3_32_32_32`	∼24%
hollywood-2009	`GB_quicksort_3_32_32_32`	∼26%

SSSP: результаты и объяснение (2/2)

Почему coPapersDBLP и hollywood-2009 — исключение

Граф	**Ср. фронт**	**Итераций**	**Среднее время итерации 2+ (Spla)**
coPapersDBLP	33 780	**16**	**∼72 ms**
hollywood-2009	118 792	**9**	**∼76 ms**

Результаты TC



TC: результаты и объяснение (1/2)

Наблюдение: **Spla доминирует на всех графах**

- Ускорение Spla над LAGraph: от **1.3×** (hollywood) до **5.2×** (rgg)
- AutoSort vs NoSort в LAGraph: разница $< 4\%$, кроме hollywood (NoSort в **1.6×** быстрее)

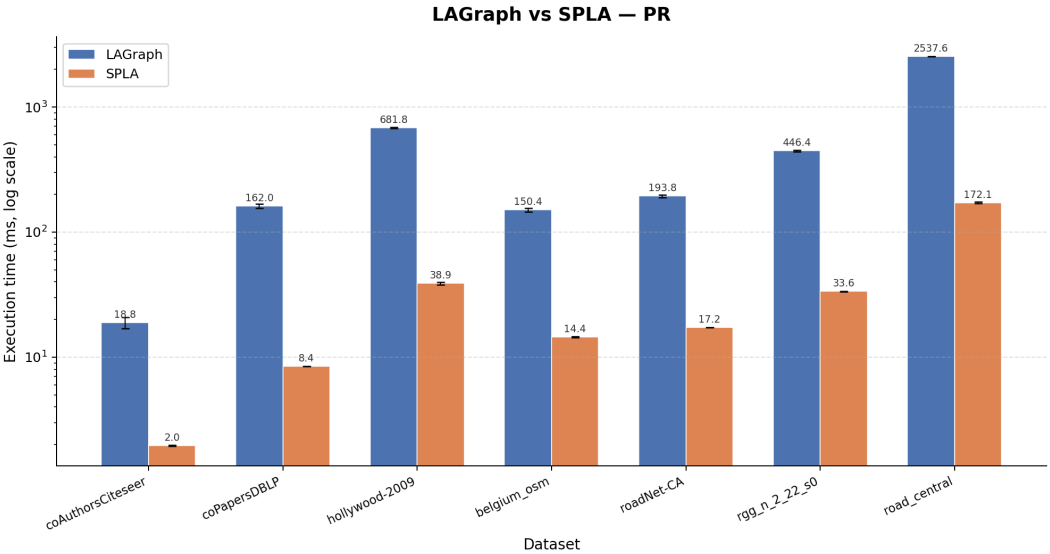
Аномалия hollywood (1.3×):

- 621M треугольников $\rightarrow$ большой объём вычислений

TC: результаты и объяснение (2/2)

Граф	Основное узкое место (Функция/Ресурс)	Доля/Overhead
Spla (GPU): Проблема очереди команд на малых графах		
coAuthorsCiteseer	OpenCL Queue Latency (vs Exec Time)	**79%**
rgg_n_2_22_s0	OpenCL Queue Latency (vs Exec Time)	**75%**
hollywood-2009	`mxmT_masked` (Compute bound)	**95% compute**
LAGraph (CPU): Сдвиг узкого места в зависимости от плотности		
coAuthorsCiteseer	`libgomp` (Parallel Overhead)	~35%
rgg_n_2_22_s0	`GB_quicksort` (Sorting)	~29%
hollywood-2009	`GB__Adot3B` (Core Compute)	~86%

Результаты PR



PR: результаты и объяснение (1/2)

Наблюдение: **Spla выигрывает на всех графах с отрывом** — самый однозначный результат среди BFS, SSSP и TC

- Ускорение Spla над LAGraph: от **9.6×** (coAuthors) до **19.2×** (coPapers)
- Минимальное ускорение — там, где граф мал

Сходимость:

- **Spla**:
  - `hollywood` сходится за **2 итерации**
  - `road_central` сходится за **1 итерацию**
- **LAGraph**:
  - `hollywood` сходится за **25 итераций**
  - `road_central` сходится за **39 итераций**

PR: результаты и объяснение (2/2)

Анализ профилей Spla

Граф (итерация)	Exec (ms)	Wall (ms)	Overhead
hollywood (iter 1)	299	1235	**76%**
hollywood (iter 2)	16.7	16.7	**∼0%**
road_central (iter 1)	184	621	**70%**

Анализ узких мест LAGraph (perf):

Граф	Доминирующая функция	Проблема
coAuthorsCiteseer	`libgomp` (62%)	OpenMP overhead на малом графе
hollywood-2009	`GB__Adot4B` (54%)	—
road_central	`GB__Adot4B` (57%)	—

Выводы (1/2)

Алгоритм	**Победитель**
BFS	Зависит от графа
SSSP	Spla (6/7)
TC	Spla (7/7)
PR	Spla (7/7)

Выводы (2/2)

Ключевые факторы преимущества GPU (Spla):

- **Большой диаметр графа**: много итераций с малым фронтиром (SSSP, BFS на дорожных графах)

Ключевые факторы преимущества CPU (LAGraph):

- **Малый диаметр + высокая плотность**: BFS на социальных графах (coPapers, hollywood) — GPU простаивает на ранних итерациях с малым фронтиром
- **GPU overhead**: OpenCL queue latency достигает 79% на малых задачах

Заключение

Ответ на исследовательский вопрос:

- GPU (Spla) превосходит CPU (LAGraph) при **прямых вычислениях** (TC) и на **разреженных графах большого диаметра** (SSSP, BFS)
- CPU (LAGraph) сохраняет преимущество только в BFS на **плотных графах с малым диаметром**